

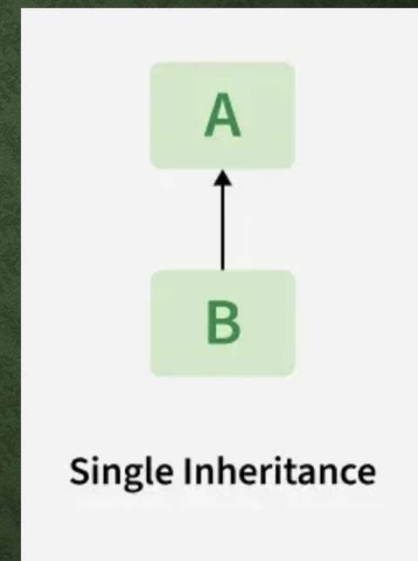
INHERITANCE

- One of the core concepts of Object-Oriented Programming (OOP)
- allows a derived class/child class) to **inherit** properties (fields) and behaviors (methods) from a base class/parent class.
- promotes **code reusability** and extensibility

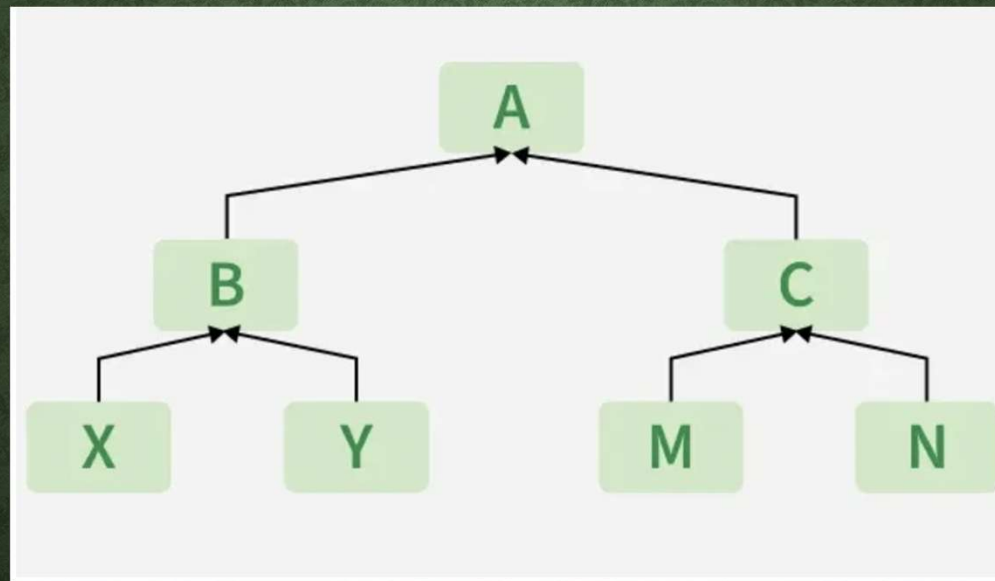
SINGLE INHERTIANCE

```
class BaseClass
{
    // Members of base class
}

class DerivedClass : BaseClass
{
    // Members of derived class
}
```



HIERARCHICAL INHERITANCE



CONSTRUCTOR

- a derived class's constructor can explicitly call a base class constructor using the **base()** keyword.

- Base class's functionalities reused by multiple derived classes, reducing code duplication.
- Derived classes can extend the functionality of the base class by **adding** new members or **overriding** existing **virtual** methods

```
class Animal {  
    public void Eat() {  
        Console.WriteLine("This animal eats food.");  
    }  
}
```

```
class Dog : Animal {  
    public void Bark() {  
        Console.WriteLine("The dog barks.");  
    }  
}
```

```
class Program {  
    static void Main() {  
        Dog d = new Dog();  
        d.Eat(); // Inherited method  
        d.Bark(); // Derived class method  
    }  
}
```


VIRTUAL

virtual keyword modifies a method, property, indexer, or event declaration in a base class, allowing it to be **overridden** in a derived class.

POLYMORPHISM

- When you have a **base** class reference pointing to a **derived** class instance, calling the virtual member will execute the derived class's overridden implementation.
- This is known as **runtime polymorphism**.


```
class Animal {  
    public virtual void Eat() {  
        Console.WriteLine("This animal eats food.");  
    }  
}  
  
class Dog : Animal {  
    public override void Eat() {  
        Console.WriteLine("This dog eats food.");  
    }  
  
    public void Bark() {  
        Console.WriteLine("The dog barks.");  
    }  
}
```

```
class Program {  
    static void Main() {  
        Dog d = new Dog();  
        d.Eat(); // Inherited method  
        d.Bark(); // Derived class method  
  
        Animal a = new Dog();  
        a.Eat(); // This dog eats food  
        //a.Bark(); // error  
    }  
}
```